



Data Wrangling

Megan Ayers
 Stat 141 | Fall 2026
 Friday, Week 2

1

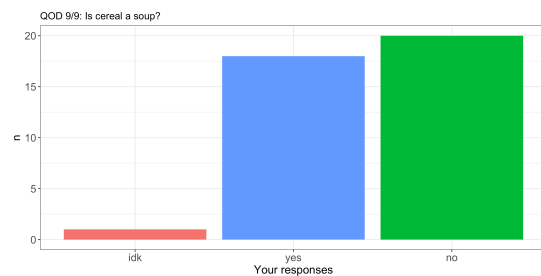
Announcement/Reminders

- First quiz will be next week during lab
 - Based closely on HW 2 exercises
 - No live coding - but maybe explaining code

2

Goals For Today

- Briefly discuss packages and the difference between base R and **dplyr**
- Learn to wrangle data, mainly with **dplyr**



3

Packages

- “Base R”: core set of built-in functions in R - no extra install required
- R is free and open source - many have contributed **packages**, like **ggplot2**, to make certain things easier to do
- **dplyr**: part of a collection of data science packages called the **tidyverse**



4

Base R vs dplyr

- Throughout the course, we will use **base R** and **dplyr** for different tasks.
- We'll try to be consistent with when we use what, but be aware that there are multiple ways to accomplish most tasks.
- Beyond this class as you work with R, I highly recommend developing skills using both base R and **dplyr**. Each has its strengths, both are widely used.

5

Data for today

```
1 dogs <- read.csv("https://data.cambridgema.gov/api/views/sckh-3xyx/rows.csv")
2
3 # Useful wrangling that we will come back to
4 dogs_top5 <- dogs %>%
5   select("Dog_Name", "Dog_Breed", "Neighborhood") %>%
6   mutate(Dog_Breed = case_when(Dog_Breed == "Mixed Breed" ~ "Mixed", TRUE ~ "Single")) %>%
7   filter(Dog_Name %in% c("Luna", "Charlie", "Lucy", "Cooper", "Rosie"))
```

6

Data for today

```
1 glimpse(dogs)
Rows: 27,774
Columns: 10
$ Dog_Name      <chr> "Poppy Grace", "Clementine", "Momo", "Maple", "MICKEY..."
$ Dog_Breed     <chr> "Pug", "Labrador Retriever", "Pomeranian", "Golden Re..."
$ Location_masked <chr> "POINT (-71.093036 42.371026)", "POINT (-71.115351 42..."
$ Latitude_masked <dbl> 42.3710, 42.3807, 42.3725, 42.3807, 42.3682, 42.3852, ...
$ Longitude_masked <dbl> -71.0930, -71.1154, -71.1084, -71.1154, -71.0791, -71...
$ Neighborhood  <chr> "Wellington-Harrington", "Baldwin", "Mid-Cambridge", "...
$ Birth.Year    <chr> "01/01/2017 12:00:00 AM", "01/01/2015 12:00:00 AM", "...
$ Gender        <chr> "Female", "Female", "Male", "Female", "Male", "Female..."
$ Expiration.Date <chr> "01/01/2021 12:00:00 AM", "01/01/2024 12:00:00 AM", "...
$ Is.Active     <chr> "false", "false", "false", "true", "false", "false", ...

1 glimpse(dogs_top5)
Rows: 909
Columns: 3
$ Dog_Name      <chr> "Lucy", "Luna", "Luna", "Charlie", "Luna", "Luna", "Luna..."
$ Dog_Breed     <chr> "Single", "Single", "Single", "Single", "Single", "Single..."
$ Neighborhood <chr> "The Port", "North Cambridge", "Mid-Cambridge", "Strawber..."
```

7

Data Wrangling: Transformations done on the data

Why wrangle the data?

To **summarize** the data.

→ To compute the mean and standard deviation of miles ridden by bikeshare users.

To **drop** missing values. (Need to be careful here!)

→ In our Lab 2, we saw that **ggplot2** drops **NA** observations before creating a graph.

To **filter** to a particular subset of the data.

→ To subset the **pdxTrees** data to just a few species types.

To **collapse** the categories of a categorical variable.

→ To go from 86 dog breeds to just mixed or single breed.

8

Data Wrangling: Transformations done on the data

Why wrangle the data?

To **arrange** the data to make it easier to display. → To sort from most common dog name to least common.

To fix how R **stores** a variable (or make a new one). → Converting quantitative variables to/from categorical variables

9

dplyr for Data Wrangling

- Five common wrangling verbs:

- `count()`
- `filter()`
- `arrange()`
- `mutate()`
- `summarize()`

- One action:

- `group_by()`

10

Motivation for “filtering”

```
1 count(dogs, Dog_Name)
# A tibble: 4,982 × 2
  Dog_Name     n
  <chr>   <int>
1 2 Chains     1
2 ABBY        12
3 ADELAIDE     9
4 AINSLEY      7
5 AJ           7
6 AJAX        8
7 ALFIE        1
8 ALI          3
9 ALICE        2
10 ALLI        6
# i 4,972 more rows
```

11

Filtering cases

We extract rows according to logical conditions about variable cases:

```
1 freddy <- filter(dogs, Dog_Name == "Freddy")
2 freddy
# A tibble: 14 × 10
  Dog_Name Dog_Breed Location_masked
  <chr>    <chr>      <dbl>
1 Freddy  German Shepherd Do... POINT (-71.128...
42.4      -71.1
2 Freddy  American Cocker Sp... POINT (-71.108...
42.4      -71.1
3 Freddy  American Cocker Sp... POINT (-71.108...
42.4      -71.1
4 Freddy  Poodle             POINT (-71.096...
42.4      -71.1
5 Freddy  Shep Mix            POINT (-71.128...
```

```
1 not_freddy <- filter(dogs, Dog_Name != "Freddy")
2 head(not_freddy)
# A tibble: 6 × 10
  Dog_Name Dog_Breed Location_masked
  <chr>    <chr>      <dbl>
1 Poppy Grace Pug          POINT (-71.093...
42.4      -71.1
2 Clementine Labrador Retriev... POINT (-71.115...
42.4      -71.1
3 Momo      Pomeranian          POINT (-71.108...
42.4      -71.1
4 Maple     Golden Retriever    POINT (-71.115...
42.4      -71.1
5 MICKEY    Yorkshire Terri... POINT (-71.079...
```



12

Filtering cases

We extract rows according to logical conditions about variable cases:

```
1 filter(dogs, Dog_Name %in% c("Freddy", "Bo"))
# A tibble: 45 × 10
  Dog_Name Dog_Breed      Location_masked Latitude_masked Longitude_masked
  <chr>    <chr>          <chr>          <dbl>          <dbl>
1 Bo      Cairn Terrier / Bo... POINT (-71.108... 42.4          -71.1
2 Freddy  German Shepherd Do... POINT (-71.128... 42.4          -71.1
3 Freddy  American Cocker Sp... POINT (-71.108... 42.4          -71.1
4 Bo      Rhodesian Ridgebac... POINT (-71.079... 42.4          -71.1
5 Bo      Mixed Breed          POINT (-71.136... 42.4          -71.1
6 Freddy  American Cocker Sp... POINT (-71.108... 42.4          -71.1
7 Bo      Cairn Terrier          POINT (-71.108... 42.4          -71.1
8 Bo      Cairn Terrier / Bo... POINT (-71.108... 42.4          -71.1
9 Bo      Mixed Breed          POINT (-71.136... 42.4          -71.1
10 Bo     American Cocker Sp... POINT (-71.108... 42.4          -71.1
# i 35 more rows
```

13

Filtering cases

We extract rows according to logical conditions about variable cases:

```
1 filter(dogs, Dog_Name == "Freddy" | Dog_Name == "Bo")
# A tibble: 45 × 10
  Dog_Name Dog_Breed      Location_masked Latitude_masked Longitude_masked
  <chr>    <chr>          <chr>          <dbl>          <dbl>
1 Bo      Cairn Terrier / Bo... POINT (-71.108... 42.4          -71.1
2 Freddy  German Shepherd Do... POINT (-71.128... 42.4          -71.1
3 Freddy  American Cocker Sp... POINT (-71.108... 42.4          -71.1
4 Bo      Rhodesian Ridgebac... POINT (-71.079... 42.4          -71.1
5 Bo      Mixed Breed          POINT (-71.136... 42.4          -71.1
6 Freddy  American Cocker Sp... POINT (-71.108... 42.4          -71.1
7 Bo      Cairn Terrier          POINT (-71.108... 42.4          -71.1
8 Bo      Cairn Terrier / Bo... POINT (-71.108... 42.4          -71.1
9 Bo      Mixed Breed          POINT (-71.136... 42.4          -71.1
10 Bo     American Cocker Sp... POINT (-71.108... 42.4          -71.1
# i 35 more rows

1 filter(dogs, Dog_Name == "Rosie" & Dog_Breed == "Schnoodle")
# A tibble: 2 × 10
  Dog_Name Dog_Breed      Location_masked Latitude_masked Longitude_masked
  <chr>    <chr>          <chr>          <dbl>          <dbl>
1 Rosie   Schnoodle POINT (-71.12829 42.38515... 42.4          -71.1
2 Rosie   Schnoodle POINT (-71.12829 42.38515... 42.4          -71.1
# i 5 more variables: Neighborhood <chr>, Birth.Year <chr>, Gender <chr>,
#   Expiration.Date <chr>, Is.Active <chr>
```

14

The pipe: %>%

The pipe operator %>% is a handy way to string multiple commands together in **dplyr**.

Instead of:

```
1 not_freddy <- filter(dogs, Dog_Name != "Freddy")
2 head(not_freddy, 3)
```

- Either method works! As you develop coding skills, consider:

... we can write:

```
1 dogs %>% filter(Dog_Name != "Freddy") %>% head(3)
```

- Readability
- Minimizing the # of defined variables
- Ease of debugging
- Personal preference

Think of the pipe operator %>% as flowing our data frame through each command.

15

Extracting variables

To extract variables, we can use the **dplyr** function **select()**

```
1 dogs_top5 %>% select("Dog_Name", "Dog_Breed") %>%
2 head()
  Dog_Name Dog_Breed
1 Lucy     Single
2 Luna     Single
3 Luna     Single
4 Charlie  Single
5 Luna     Single
6 Luna     Single
```

To extract just one variable for a one-time use, it can be easier to just use **\$**

```
1 head(dogs_top5$Dog_Name)
[1] "Lucy" "Luna" "Luna" "Charlie" "Luna" "Luna"
```

16

Arranging cases

Use `arrange()` to change the order of cases in a data frame

```
1 dogs_top5 %>%
2   arrange(Dog_Name)

  Dog_Name Dog_Breed Neighborhood
1 Charlie Single Strawberry Hill
2 Charlie Single Neighborhood Nine
3 Charlie Single The Port
4 Charlie Single Mid-Cambridge
5 Charlie Single East Cambridge
6 Charlie Single Cambridgeport
7 Charlie Single Riverside
8 Charlie Single East Cambridge
9 Charlie Single Baldwin
10 Charlie Single North Cambridge
11 Charlie Single Mid-Cambridge
12 Charlie Single North Cambridge
13 Charlie Mixed Strawberry Hill
```

17

Creating a new variable

We use `mutate()` to create new variables, or overwrite existing ones

```
1 head(dogs_top5)
  Dog_Name Dog_Breed Neighborhood
1 Lucy Single The Port
2 Luna Single North Cambridge
3 Luna Single Mid-Cambridge
4 Charlie Single Strawberry Hill
5 Luna Single West Cambridge
6 Luna Single Strawberry Hill

1 dogs_top5 %>%
2   mutate(animal = "dog",
3          Dog_Name = toupper(Dog_Name)) %>%
4   head()

  Dog_Name Dog_Breed Neighborhood animal
1 LUCY Single The Port dog
2 LUNA Single North Cambridge dog
3 LUNA Single Mid-Cambridge dog
4 CHARLIE Single Strawberry Hill dog
5 LUNA Single West Cambridge dog
6 LUNA Single Strawberry Hill dog
```

18

New Data Setting: Bureau of Labor Statistics (BLS) Consumer Expenditure Survey

BLS Mission: "Measures labor market activity, working conditions, price changes, and productivity in the U.S. economy to support public and private decision making."

Data: Last quarter of the 2016 BLS Consumer Expenditure Survey.

```
1 ce_raw <- read.csv("data/fmli.csv", na = c("NA", ""))
2 str(ce_raw)

'data.frame': 6301 obs. of 51 variables:
 $ NEWID : int 3324174 3324204 3324214 3324244 3324274 3324284 3324294 3324304 3324324 3324334 ...
 $ PRINEARN: int 1 1 1 1 2 1 1 1 2 1 ...
 $ FINLWT21: num 25985 6581 26208 18078 26112 ...
 $ FINCBTAX: int 116920 200 117000 0 2000 942 0 91000 95000 40037 ...
 $ BLS_URBN: int 1 1 1 1 1 1 1 2 1 ...
 $ POPSIZE : int 2 3 4 2 2 2 1 2 5 2 ...
 $ EDUC_REF: int 16 15 16 15 14 11 10 13 12 12 ...
 $ EDUC2 : int 15 15 13 NA NA NA NA 15 15 14 ...
 $ AGE_REF : int 63 50 47 37 51 63 77 37 51 64 ...
 $ AGE2 : int 50 47 46 NA NA NA NA 36 53 67 ...
 $ SEX_REF : int 1 1 2 1 2 1 2 1 2 ...
 $ SEX2 : int 2 2 1 NA NA NA NA 2 2 1 ...
 $ REF_RACE: int 1 4 2 1 1 1 1 1 1 1 ...
 $ RACE2 : int 1 4 1 NA NA NA NA 1 1 1 ...
 $ HISP_REF: int 2 2 2 2 1 1 2 2 ...
 $ HISP2 : int 2 2 1 NA NA NA NA 2 2 ...
 $ FAM_TYPE: int 3 4 1 8 9 8 3 1 1 ...
 $ MARITAL1: int 1 1 1 5 3 3 2 1 1 1 ...
 $ REGION : int 4 4 3 4 4 3 4 1 3 2 ...
```

19

Wrangling CE Data

Want to better understand a family's income and expenditures

```
1 ce <- ce_raw %>% select("NEWID", "PRINEARN", "FINCBTAX", "BLS_URBN", "HIGH_EDU", "TOTEXPCQ", "IRAX")
2 dim(ce)

[1] 6301 7
```

Variables:

- **NEWID:** ID for the household
- **PRINEARN:** ID for which member of the household is the principal earner
- **FINCBTAX:** Final income before taxes for the year
- **BLS_URBN:** 1 = urban, 2 = rural
- **HIGH_EDU:** Highest education in the household. 00 = Never attended, 10 = Grades 1-8, 11 = Grades 9-12, no degree, 12 = High school graduate, 13 = Some college, no degree, 14 = Associates degree, 15 = Bachelor's degree, 16 = Masters, Professional/doctorate degree
- **TOTEXPCQ** = Total household expenditures for the current **quarter**
- **IRAX** = Total in retirement funds

20

Wrangling CE Data

Q: What is this code doing?

```
1 ce <- ce %>%
2   mutate(YEARLY_EXP = TOTEXPCQ*4)
3 ce
```

	NEWID	PRINEARN	FINCBTAX	BLS_URBN	HIGH_EDU	TOTEXPCQ	IRAX	YEARLY_EXP
1	3324174	1	116920	1	16	0.0000	1000000	0.0000
2	3324204	1	200	1	15	0.0000	10000	0.0000
3	3324214	1	117000	1	16	0.0000	0	0.0000
4	3324244	1	0	1	15	0.0000	NA	0.0000
5	3324274	2	2000	1	14	0.0000	NA	0.0000
6	3324284	1	942	1	11	0.0000	0	0.0000
7	3324294	1	0	1	10	0.0000	0	0.0000
8	3324304	1	91000	1	15	0.0000	15000	0.0000
9	3324324	2	95000	2	15	0.0000	NA	0.0000
10	3324334	1	40037	1	14	0.0000	477000	0.0000
11	3324394	2	109000	1	12	0.0000	NA	0.0000
12	3324404	1	4104	1	12	0.0000	NA	0.0000
13	3324424	1	0	1	12	0.0000	NA	0.0000

(Reminder that **TOTEXPCQ** is total quarterly expenditures.)

Using logical operators

Q: What is this code doing?

```
1 ce_sub <- ce %>%
2   filter(YEARLY_EXP > 0, BLS_URBN == 1, HIGH_EDU != "00")
3 ce_sub
```

	NEWID	PRINEARN	FINCBTAX	BLS_URBN	HIGH_EDU	TOTEXPCQ	IRAX	YEARLY_EXP
1	3335204	1	37000	1	14	2491.9167	0	9967.6668
2	3335214	1	103000	1	16	6128.3334	NA	24513.3336
3	3335224	1	14686	1	13	1071.8000	NA	4287.2000
4	3335244	2	33396	1	12	1630.0000	0	6520.0000
5	3335264	1	0	1	13	3213.3667	NA	12853.4668
6	3335274	1	0	1	15	4673.5833	0	18694.3332
7	3335294	1	745136	1	16	8693.3727	280000	34773.4908
8	3335304	1	36000	1	16	3733.1667	NA	14932.6668
9	3335314	2	45000	1	15	3627.2500	3000	14509.0000
10	3335334	1	20862	1	13	802.3333	0	3209.3332
11	3335344	1	0	1	16	7670.7833	NA	30683.1332
12	3335354	2	47662	1	13	4179.8833	400000	16719.5332
13	3335374	1	10560	1	15	2728.3333	NA	10913.3332

Using logical operators

Q: What is this code doing?

```
1 ce_sub <- ce %>%
2   filter(YEARLY_EXP > 0, (BLS_URBN == 1 | HIGH_EDU != "00"))
3 ce_sub
```

	NEWID	PRINEARN	FINCBTAX	BLS_URBN	HIGH_EDU	TOTEXPCQ	IRAX	YEARLY_EXP
1	3335204	1	37000	1	14	2491.9167	0	9967.6668
2	3335214	1	103000	1	16	6128.3334	NA	24513.3336
3	3335224	1	14686	1	13	1071.8000	NA	4287.2000
4	3335244	2	33396	1	12	1630.0000	0	6520.0000
5	3335264	1	0	1	13	3213.3667	NA	12853.4668
6	3335274	1	0	1	15	4673.5833	0	18694.3332
7	3335294	1	745136	1	16	8693.3727	280000	34773.4908
8	3335304	1	36000	1	16	3733.1667	NA	14932.6668
9	3335314	2	45000	1	15	3627.2500	3000	14509.0000
10	3335334	1	20862	1	13	802.3333	0	3209.3332
11	3335344	1	0	1	16	7670.7833	NA	30683.1332
12	3335354	2	47662	1	13	4179.8833	400000	16719.5332
13	3335374	1	10560	1	15	2728.3333	NA	10913.3332

Using logical operators

Q: Which version leaves more cases in the data frame?

```
1 ce_sub <- ce %>%
2   filter(YEARLY_EXP > 0, BLS_URBN == 1, HIGH_EDU != "00")
```

```
1 ce_sub <- ce %>%
2   filter(YEARLY_EXP > 0, (BLS_URBN == 1 | HIGH_EDU != "00"))
```

Using logical operators

Q: Which version leaves more cases in the data frame?

```
1 ce_sub <- ce %>%
2   filter(YEARLY_EXP > 0, BLS_URBN == 1, HIGH_EDU != "00")
3 dim(ce_sub)
[1] 3954    8
```

```
1 ce_sub <- ce %>%
2   filter(YEARLY_EXP > 0, (BLS_URBN == 1 | HIGH_EDU != "00"))
3 dim(ce_sub)
[1] 4178    8
```

25

Recoding Variables with `case_when()`

```
1 count(ce, BLS_URBN)
  BLS_URBN    n
1         1 5952
2         2  349
```

Q: What is this code doing?

```
1 ce <- ce %>%
2   mutate(BLS_URBN = case_when(BLS_URBN == 1 ~ "Urban",
3                               BLS_URBN == 2 ~ "Rural",
4                               TRUE ~ NA))
5
6 count(ce, BLS_URBN)
  BLS_URBN    n
1     Rural  349
2     Urban 5952
```

26

Creating Variables with `case_when()`

```
1 ce %>%
2   mutate(HIGH_EDU = as.numeric(HIGH_EDU)) %>%
3   count(HIGH_EDU)
  HIGH_EDU    n
1         0     8
2        10   110
3        11   302
4        12 1272
5        13 1297
6        14   714
7        15 1528
8        16 1070
```

```
1 ce <- ce %>%
2   mutate(HIGH_EDU2 = case_when(HIGH_EDU <= 11 ~ "Less than high school degree",
3                               HIGH_EDU <= 13 ~ "High school degree",
4                               HIGH_EDU >= 14 ~ "College degree",
5                               TRUE ~ NA))
6
7 count(ce, HIGH_EDU2)
  HIGH_EDU2    n
1 College degree 3312
2 High school degree 2569
3 Less than high school degree 420
```

27

Variable Names

Sometimes datasets come with terrible variable names.

```
1 names(ce)
[1] "NEWID"    "PRINEARN"  "FINCBTAX"  "BLS_URBN"  "HIGH_EDU"
[6] "TOTEXPCQ" "IRAX"     "YEARLY_EXP" "HIGH_EDU2"
```

We can fix that with `rename()`.

```
1 ce <- ce %>%
2   rename(INCOME = FINCBTAX)
```

28

Handling Missing Data

- (Time check - possibly skip and review independently)

Want to compute mean income and mean retirement funds.

```
1 mean(ce$INCOME)
[1] 62480.12

1 mean(ce$IRAX)
[1] NA

1 ce_aggressive <- na.omit(ce_raw)
2 ce_aggressive

[1] NEWID PRINEARN FINLWT21 FINCBTAX BLS_URBN POPSIZE
EDUC_REF EDUCA2
[9] AGE_REF AGE2 SEX_REF SEX2 REF_RACE RACE2
HISP_REF HISP2
[17] FAM_TYPE MARITAL1 REGION SMSASTAT HIGH_EDU EHOUNSGC
TOTEXPCQ FOODCQ
[25] TRANSCQ HEALTHCQ ENTERTCQ EDUCACQ TOBACCQ STUFINX
IRAX CUTENURE
[33] FAM_SIZE VEHQ ROOMSQ INC_HRS1 INC_HRS2 EARNCOMP
NO_EARNR OCCUCOD1
[41] OCCUCOD2 STATE DIVISION TOTXEST CREDFINX CREDITB
CREDITX BUILDING
[49] ST_HOUS INT_PHON INT_HOME
<0 rows> (or 0-length row.names)
```

- Q: Guesses about what's going on here?

29

Handling Missing Data

```
1 ce_moderate <- ce %>%
2 filter(!is.na(IRAX), !is.na(INCOME))
3
4 mean(ce_moderate$INCOME)
[1] 57771.14
1 mean(ce_moderate$IRAX)
[1] 91075.09
```

Or, to be very conservative about preserving data:

```
1 mean(ce$INCOME, na.rm = TRUE)
[1] 62480.12
1 mean(ce$IRAX, na.rm = TRUE)
[1] 91075.09
```

Q: Why does the mean income value change between these approaches, even though there are no NA income values?

A: The moderate approach still drops **INCOME** values if a row is missing **IRAX**.

30

Summarizing within groups using `group_by()` and `summarize()`

```
1 ce_moderate %>%
2 group_by(BLS_URBN) %>%
3 summarize(mean_IRAX = mean(IRAX))

# A tibble: 2 × 2
  BLS_URBN mean_IRAX
  <chr>      <dbl>
1 Rural    37008.
2 Urban    94512.

1 ce_moderate %>%
2 group_by(BLS_URBN, HIGH_EDU2) %>%
3 summarize(mean_IRAX = mean(IRAX))

# A tibble: 6 × 3
# Groups:   BLS_URBN [2]
  BLS_URBN HIGH_EDU2 mean_IRAX
  <chr>      <chr>      <dbl>
1 Rural    College degree 105148.
2 Rural    High school degree 15543.
3 Rural    Less than high school degree 0
4 Urban    College degree 168767.
5 Urban    High school degree 30533.
6 Urban    Less than high school degree 8270.
```

31

Naming Wrangled Data

When I make a new data frame, what name should I give it? Importantly, should I **write over my original dataframe** or should I **save a new dataframe**?

- Advice:
 - Is your new data frame structurally different? If so, give it a **new name**.
 - Are you removing values you will need for a future analysis within the same document? If so, give it a **new name**.
 - Are you just adding to or cleaning the data? If so, then **write over** the original.

32

Sage Advice from ModernDive

“Crucial: Unless you are very confident in what you are doing, it is worthwhile not starting to code right away. Rather, first sketch out on paper all the necessary data wrangling steps not using exact code, but rather high-level pseudocode that is informal yet detailed enough to articulate what you are doing. This way you won’t confuse what you are trying to do (the algorithm) with how you are going to do it [(writing code)].”

33

In-Class Exercise: Data wrangling pseudocode

Anonymized data from a 2017 study on the effect of the higher education system on upward mobility (Chetty et al. 2017).

```
1 load("data/colleges.Rdata")
2 head(colleges[, c("region", "state", "tier_name", "name", "mr_kq5_pq1")])

# A tibble: 6 × 5
  region state tier_name      name      mr_kq5_pq1
  <fct> <chr> <fct>      <chr>      <dbl>
1 3 TX Selective private Abilene Christian Univers... 0.0144
2 3 GA Nonselective 4-year public Abraham Baldwin Agricultu... 0.0149
3 4 CO Selective public Adams State University 0.0188
4 1 NY Selective private Adelphi University 0.0326
5 2 MI Selective private Adrian College 0.00956
6 3 FL Nonselective 4-year private Adventist University Of H... 0.0379
```

- **region:** 1 for Northeast, 2 for Midwest, 3 for South, and 4 for West
- **tier_name:** Tier defined by selectivity and type
- **mr_kq5_pq1:** Mobility rate, top 20% of the income distribution.
- **state:** State name

In groups of 3-4, on paper or the board, write pseudocode to answer: what states in the west have the highest average mobility rate across their selective private and selective public colleges?

07:00

34

Recap

- Five fundamental wrangling verbs from the **dplyr** package:
 - **count()**: reduces the data set to counts of categorical variables
 - **filter()**: subsets the data frame to rows that meet logical conditions
 - **arrange()**: sorts the data frame
 - **mutate()**: creates new columns in the data frame (can be combined with arithmetic, **case_when()**, and more)
 - **summarize()**: reduces the data set to summary statistics
- One **dplyr** action:
 - **group_by()**:
 - Tells **R**: perform the following operation separately for levels of a categorical variable
 - Mostly used in combination with **summarize()** (sometimes with **mutate()**)
- Use the pipe **%>%** between function calls to perform several data wrangling steps at once

35

Next Time

- Shifting gears into probability

36